

Committee Automator

Setup & Handoff Guide

This document explains what the Committee Automator pipeline does, the exact folder structure the code expects, every prerequisite, and step-by-step setup instructions for a new machine — including the NotebookLM authentication step, which is the least standard part of the stack.

1. What This Does

The pipeline turns student article submissions (.eml files) into ready-to-post LinkedIn posters, automatically:

```
.eml file
-> parse text + extract photo
-> Gemini writes a title/summary
-> NotebookLM generates a background illustration
-> Python composites a poster (photo + title + byline)
-> poster uploaded, row appended to a Google Sheet
```

Pipeline modules

File	Responsibility
batch_run.py	Entry point. Loops every .eml in Article EMLs/, runs the full pipeline, moves finished files to Completed/.
eml_parser.py	Extracts plain text, author name, LinkedIn URL, and photo attachment from an .eml.
ai_services.py	Calls Gemini for title/summary/image-prompt, and the nlm CLI (NotebookLM) for background art.
composer.py	Pillow + OpenCV: face-centered circular photo crop, banner/title/byline layout, final 1080x1080 poster.
operations.py	Google Sheets append, ImgBB upload; optional Drive upload and LinkedIn posting helpers.
main.py	Alternate single-file pipeline runner (used for one-off/manual runs); includes a mock-poster fallback.

2. Folder Structure & Naming Convention

The names below are hardcoded relative paths in the code — not configurable via environment variables. Preserve them exactly, at the project root, if replicating this setup (including via a coding agent).

```
committee-automator/
├── batch_run.py           entry point - run this
├── eml_parser.py
├── ai_services.py
├── composer.py
├── operations.py
├── main.py
├── requirements.txt
├── .env.example           copy to .env, fill in keys
└── credentials.example.json copy to credentials.json
```

— assets/	
— haarcascade_frontalface_default.xml	
— Roboto-Bold.ttf	
— Article EMLs/	DROP .eml FILES HERE
— Completed/	processed files land here
— output/	generated photos/posters

Name (exact)	Why it must stay exact
Article EMLs	Capital A/E, with a space — batch_run.py joins this literal string to find files to process.
Article EMLs/Completed	Where finished .eml files are moved so re-runs don't reprocess them.
output	Lowercase. Generated photos and posters land here; auto-created if missing.
assets	Lowercase. Fixed relative paths to the font and face-detection model used by composer.py.
credentials.json	Exact filename, project root — operations.py opens this literal name.
.env	Project root, loaded by python-dotenv; must sit next to batch_run.py.

3. Prerequisites

System

- Python 3.11+ (developed/tested on 3.14)
- macOS, Linux, or Windows — no OS-specific code, but the NotebookLM auth step needs a real desktop browser
- Node.js is NOT required — earlier prototypes used an npm-based NotebookLM MCP server; the current pipeline does not

Accounts / API keys

Service	Purpose	Where to get it
Google Gemini	Title/summary/image-prompt generation	aistudio.google.com/apikey (free tier: 5 req/min)
Google Cloud service account	Writing to the Google Sheet	console.cloud.google.com → IAM & Admin → Service Accounts
A Google Sheet	Where results get logged	Any sheet you own, shared with the service account's client_email as Editor
ImgBB	Free public image hosting for the poster	api.imgbb.com (free API key, no card)
Personal Google account w/ NotebookLM	Generates background art	See Section 5 — this is the unusual one

4. Install

```
git clone <this-repo-url>
cd committee-automator
```

```
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

cp .env.example .env
cp credentials.example.json credentials.json
```

Fill in `.env` with your real `GEMINI_API_KEY`, `SHEET_ID`, and `IMGBB_API_KEY`. Replace `credentials.json` with your actual downloaded service-account key (same filename, project root).

Never commit `.env` or `credentials.json` with real values — both are in `.gitignore` for this reason.

5. Setting Up NotebookLM (the `nml` CLI)

The background-art step doesn't call a conventional API — NotebookLM has no public API. It works by driving your own logged-in NotebookLM session through a CLI tool (`notebooklm-mcp-cli`, binary name `nml`). Each person running this pipeline needs their own Google account authenticated locally on their own machine — this step cannot run unattended on a shared server, and credentials cannot be shared like an API key.

5.1 Install `nml`

Recommended: `pipx`, which keeps it isolated from your system Python.

```
# macOS
brew install pipx
pipx ensurepath

# then, any OS with pipx installed:
pipx install notebooklm-mcp-cli
```

Alternative: `pip install --user notebooklm-mcp-cli` (or inside a `venv`), but `pipx` is recommended to avoid colliding with this project's own `venv` packages.

Confirm installation:

```
nml --version
```

5.2 Log in

```
nml login
```

This opens a real Chrome window. Sign into the Google account you want NotebookLM to run as. Once you land on `notebooklm.google.com` successfully, cookies are saved locally — you won't need to log in again on that machine.

Verify:

```
nml notebook list
```

This should list your NotebookLM notebooks (or an empty list, not an auth error) if login succeeded.

5.3 What the pipeline calls

`ai_services.py` shells out to `nml` as a subprocess (see the `_nml()` helper in that file) to:

- `nml notebook create` — one notebook per article

- `nlm source add --file ...` — uploads the article text as a source
- `nlm infographic create` — generates the background art, using one of a fixed set of style/prompt combinations tuned to produce zero-text, LinkedIn-appropriate illustrations (see `NLM_VARIANTS` in `ai_services.py`)
- `nlm studio status` — polls until the artifact is ready
- `nlm download infographic` — downloads the PNG
- `nlm notebook delete` — by default, runs automatically once the images are downloaded, so the notebook doesn't linger in your NotebookLM library

Cleanup is best-effort: a delete failure is logged, not fatal, and the pipeline keeps going (that notebook is just left behind). To keep a notebook around instead — e.g. to generate more variants against the same uploaded source later without re-uploading it — call `generate_background_candidates(..., cleanup_notebook=False)`.

5.4 Known limitation: rate limits

NotebookLM enforces an undocumented rate limit on infographic generation. In practice, roughly a dozen-plus generations in a short window triggers a "Rate limited (RESOURCE_EXHAUSTED)" error that does not clear in the "1-2 minutes" the error message claims — it behaves more like an hourly/daily cap. `ai_services.py` already retries with backoff (several minutes per attempt), but for large batches, expect to occasionally need to stop and resume later. There is no way around this without switching to a different (paid) image-generation backend.

6. Running It

Drop .eml files into Article EMLs/, then:

```
python batch_run.py
```

Each article gets parsed, generates its own NotebookLM notebook and artwork, gets composited into a poster, uploaded to ImgBB, and appended as a row to your Google Sheet — then the source .eml moves to Article EMLs/Completed/.

`main.py` is an alternate single-file entry point (`run_pipeline(path)`) that also has LinkedIn-posting and Google Drive upload wired in, both optional/unused by default.

7. Troubleshooting

Symptom	Likely cause / fix
ModuleNotFoundError: No module named 'cv2'	Not running inside the venv, or <code>opencv-python-headless</code> didn't install — re-run <code>pip install -r requirements.txt</code> inside the activated venv.
Poster background is a plain dark rectangle	The NotebookLM image download or generation failed; <code>ai_services.py</code> prints "[NotebookLM Generation Failed: ...]" and falls back to a mock background — check the console output above that line.
Photo circle shows the wrong part of the face	OpenCV's Haar cascade occasionally false-positives on a small region (e.g. a tie knot). <code>get_face_bbox()</code> in <code>composer.py</code> filters detections smaller than 15% of the image's shorter dimension for this reason; if it persists, the source photo may need a manual crop.
<code>gspread.exceptions.APIError: [503] / [502]</code>	Transient Google API hiccup — just retry.
NotebookLM step hangs or errors "Rate limited"	See Section 5.4.

8. What's Intentionally Not in This Repo

This repo contains only the working pipeline. It does not include: real `.env` / `credentials.json` values, any actual student submissions or generated posters (personal data), or the various one-off experiment/debug scripts from earlier development — abandoned Hugging Face FLUX attempts, Pollinations.ai tests, several earlier NotebookLM-MCP integration attempts that didn't work out, sheet-inspection scripts, and similar. None of that is part of the working system, and including it would only add confusion to a fresh setup.